

OSSP PRACTICAL PROGRAMQ1: -

1. Develop a C program that demonstrates how a Linux operating system executes a command entered by a user that:

1. Accept a Linux command as input.
2. Create a child process using fork ().
3. Execute the command in the child process using an appropriate exec () system call.
4. Allow the parent process to wait for the child using wait.
5. Display the Process ID (PID) of both parent and child processes.

CODE: -

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
int main() {
    char command[100];
    pid_t pid;
    printf("Enter a Linux command: ");
    scanf("%99s", command);
    pid = fork();
    if (pid < 0) {
        printf("Fork failed!\n");
        return 1;
    }
    if (pid == 0) {
        printf("Child Process\n");
        printf("Child PID: %d\n", getpid());
        printf("Parent PID: %d\n", getppid());
        execlp(command, command, NULL);
        printf("Command execution failed!\n");
        exit(1);
    }
```

```
}  
else {  
    printf("Parent Process\n");  
    printf("Parent PID: %d\n", getpid());  
    printf("Child PID: %d\n", pid);  
    wait(NULL);  
    printf("Child process completed.\n");  
}  
return 0;  
}
```

OUTPUT: -

```
[apple@apples-MacBook-Pro-2 OSSP % nano q1.c  
[apple@apples-MacBook-Pro-2 OSSP % ./q1  
Enter a Linux command: ls  
  
Parent Process  
Parent PID : 2917  
Child PID : 2918  
  
Child Process  
Child PID : 2918  
Parent PID: 2917  
a.out          p3          p6          p9          q2.c  
copy.txt       p3.c        p6.c        p9.c        sample.txt  
p1             p4          p7          practical   skill  
p1.c           p4.c        p7.c        q1  
p2             p5          p8          q1.c  
p2.c           p5.c        p8.c        q2  
  
Child process completed.
```

OSSP PRACTICAL PROGRAM2: -

Develop a C program that uses the system calls open(), read(), write(), and close() to copy the contents of one file to another. Explain how control transitions between user space and kernel space during execution.

Use the strace utility to trace all system calls generated by the command cat sample.txt. Analyze the sequence of system calls and identify the kernel services involved.

Code: -

```
#include <stdio.h>

#include <stdlib.h>

#include <fcntl.h>

#include <unistd.h>


int main() {

    int source, destination;

    char buffer[1024];

    ssize_t bytesRead;


    source = open("source.txt", O_RDONLY);


    if (source == -1) {

        perror("Error opening source file");

        return 1;

    }
```

```
destination = open("destination.txt",
    O_WRONLY | O_CREAT | O_TRUNC,
    0644);

if (destination == -1) {
    perror("Error opening destination file");
    close(source);
    return 1;
}

while ((bytesRead = read(source, buffer, sizeof(buffer))) > 0) {
    if (write(destination, buffer, bytesRead) != bytesRead) {
        perror("Error writing to destination file");
        close(source);
        close(destination);
        return 1;
    }
}

if (bytesRead == -1) {
    perror("Error reading source file");
} else {
    printf("File copied successfully!\n");
}

close(source);
```

```

close(destination);

return 0;

}

```

OUTPUT: -

The screenshot shows a terminal window titled "OSSP — nano q2.c — 80x24". The window contains the source code for a C program that copies the contents of a source file to a destination file. The code includes headers for stdio, fcntl, unistd, and stdlib, and defines a BUFFER_SIZE of 1024. The main function prompts the user for source and destination file names, reads the source file in chunks, and writes it to the destination file. Below the code, the terminal output shows the user running the program, entering "sample.txt" as the source and "copy.txt" as the destination, and seeing the message "File copied successfully!". Finally, the user runs "cat copy.txt" and sees the output "Hello OSSP Lab" and "This is Question 2.".

```

UW PICO 5.09                               File: q2.c                               Modified

#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdlib.h>

#define BUFFER_SIZE 1024

int main() {
    char source[100], destination[100];
    char buffer[BUFFER_SIZE];
    int sourceFile, destFile;
    ssize_t bytesRead;

    printf("Enter source file name: ");
    scanf("%99s", source);

    printf("Enter destination file name: ");
    scanf("%99s", destination);

    ^G Get Help   ^O WriteOut    ^R Read File   ^Y Prev Pg     ^K Cut Text    ^C Cur Pos
    ^X Exit       ^J Justify     ^W Where is   ^V Next Pg     ^U UnCut Text  ^T To Spell

apple@apples-MacBook-Pro-2 OSSP % nano q2.c
apple@apples-MacBook-Pro-2 OSSP % ./q2
Enter source file name: sample.txt
Enter destination file name: copy.txt
File copied successfully!
apple@apples-MacBook-Pro-2 OSSP % cat copy.txt
Hello OSSP Lab
This is Question 2.
apple@apples-MacBook-Pro-2 OSSP %

```

OSSP PRACTICAL PROGRAM 3: -

Develop a C program using fork() that creates a parent and child process. Display the Process ID (PID), Parent Process ID (PPID), and process states at different stages of execution.

Design an experiment to observe process state transitions (Ready, Running, Waiting, Terminated) using Linux monitoring tools such as ps, top, and /proc. Document the observations.

Code: -

```
#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

#include <sys/types.h>

#include <sys/wait.h>

int main() {

    pid_t pid;

    printf("Parent process started\n");

    printf("Parent PID : %d\n", getpid());

    printf("Parent PPID : %d\n\n", getppid());

    pid = fork();

    if (pid < 0) {

        perror("fork failed");

        return 1;

    }

    if (pid == 0) {

        printf("----- Child Process ----- \n");
```

```
    printf("Child PID : %d\n", getpid());

    printf("Child PPID : %d\n", getppid());

    printf("Child is running...\n");

    sleep(5);

    printf("Child process terminated.\n");

    exit(0);
}

else {

    printf("----- Parent Process ----- \n");

    printf("Parent PID : %d\n", getpid());

    printf("Child PID : %d\n", pid);

    printf("Parent is waiting for child...\n");

    wait(NULL);

    printf("Child process has terminated.\n");

    printf("Parent process terminated.\n");

}

return 0;
}
```


OUTPUT: -

```
practical — nano practicalq3.c — 80x24
UW PICO 5.09 File: practicalq3.c

#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

#include <sys/types.h>

#include <sys/wait.h>

int main() {

    pid_t pid;

    printf("Parent process started\n");

    printf("Parent PID : %d\n", getpid());

    printf("Parent PPID : %d\n\n", getppid());

^G Get Help ^O WriteOut ^R Read File ^Y Prev Pg ^K Cut Text ^C Cur Pos
^X Exit ^J Justify ^W Where is ^V Next Pg ^U UnCut Text ^T To Spell

apple@apples-MacBook-Pro-2 practical % nano practicalq3
apple@apples-MacBook-Pro-2 practical % nano pratical q3.c
apple@apples-MacBook-Pro-2 practical % nano practical.c
apple@apples-MacBook-Pro-2 practical % nano practicalq3.c
apple@apples-MacBook-Pro-2 practical % gcc practicalq3.c -o practicq3
apple@apples-MacBook-Pro-2 practical % ./practicq3
Parent process started
Parent PID : 3125
Parent PPID : 2807

----- Parent Process -----
Parent PID : 3125
Child PID : 3126
Parent is waiting for child...
----- Child Process -----
Child PID : 3126
Child PPID : 3125
Child is running...
Child process terminated.
Child process has terminated.
Parent process terminated.
```


OSSP PRACTICAL PROGRAM Q4: -

Write a C program where a parent process creates multiple child processes and synchronizes their completion using wait() and waitpid(). Compare the behavior of both functions.

Extend the program to demonstrate how improper synchronization can lead to zombie processes. Use appropriate system calls to prevent zombie processes.

Code: -

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
int main() {
    pid_t pid[3];
    int i, status;
    for (i = 0; i < 3; i++) {
        pid[i] = fork();
        if (pid[i] < 0) {
            perror("fork failed");
            exit(1);
        }
        if (pid[i] == 0) {
            printf("Child %d started\n", i + 1);
            printf("PID : %d\n", getpid());
            printf("PPID : %d\n", getppid());
            sleep(i + 2);
            printf("Child %d finished\n\n", i + 1);
            exit(0)
        }
    }
    printf("Parent PID: %d\n\n", getpid());
    printf("Using wait():\n");
    for (i = 0; i < 2; i++) {
        wait(&status);
```

```
    printf("A child process terminated.\n");  
}  
printf("\nUsing waitpid():\n");  
waitpid(pid[2], &status, 0);  
printf("Child with PID %d terminated.\n", pid[2]);  
printf("\nAll child processes completed.\n");  
return 0;  
}
```

OUTPUT: -